Міністерство Освіти і Науки України

Національний Університет “Львівська Політехніка”

Кафедра ЕОМ

МНОЖЕННЯ МАТРИЦЬ НА СТРУКТУРІ 2D РЕШІТКА ЗАСОБАМИ MPI

Методичні вказівки
до лабораторної роботи з курсу “Паралельні та розподілені обчислення”
для студентів спеціальності F7 “Комп’ютерна інженерія”

Затверджено
на засіданні кафедри
”Електронні обчислювальні машини”
Протокол № _ від 2026 року

Львів – 2026

МЕТА РОБОТИ. Вивчити можливості множення матриць на структурі 2D решітка. Набути навиків такого множення засобами MPI.

ТЕОРЕТИЧНІ ВІДОМОСТІ

Множення матриць є однією з найпоширеніших операцій у наукових обчисленнях, комп'ютерній графіці, машинному навчанні та моделюванні фізичних процесів. Для великих розмірів матриць виконання цієї операції на одному процесорі може бути дуже повільним, тому застосовуються паралельні алгоритми множення матриць.

Одним з ефективних підходів до паралельного множення матриць є використання структури 2D решітка процесів.

Нехай задано дві матриці:

$$A_{m\times k}, B_{k\times m}.$$

Результатом множення буде матриця:

$$C_{m\times n}$$

елементи якої обчислюються за формулою:

$$C_{ij} = \sum_{t=1}^{k} A_{it} * B_{tj}$$

Цей алгоритм містить три вкладені цикли і має часову складність $O(n^3)$ для квадратних матриць розміру $n\times n$.

Для пришвидшення обчислень операцію множення матриць часто виконують паралельно на багатьох процесах.

У паралельному алгоритмі обчислення розподіляються між декількома процесами. Кожен процес виконує обчислення лише частини матриці C. Основними задачами при цьому є:

- розподіл даних між процесами;
- організація обміну даними між процесами;
- збір результатів обчислення.

Одним із найбільш ефективних способів організації процесів є структура 2D решітка.

У цьому підході процеси організовуються у вигляді двовимірної сітки розміру

$$p\times q$$

де $p*q$ – загальна кількість процесів.

Наприклад, для 8 процесів можлива решітка 2 × 4:

P(0,0) P(0,1) P(0,2) P(0,3)

P(1,0) P(1,1) P(1,2) P(1,3)

Або решітка решітка 4 × 2:

P(0,0) P(0,1)

P(1,0) P(1,1)

P(2,0) P(2,1)

P(3,0) P(3,1)

Кожен процес має координати (i, j), де i – номер рядка решітки, а j – номер стовпця решітки.

Матриці при цьому розбиваються на смуги. Матриця A ділиться по рядках, а матриця B – по стовпцях.

Для решітки 2 × 4 матриця A ділиться на 2 смуги по рядках, а матриця B ділиться на 4 смуги по стовпцях:

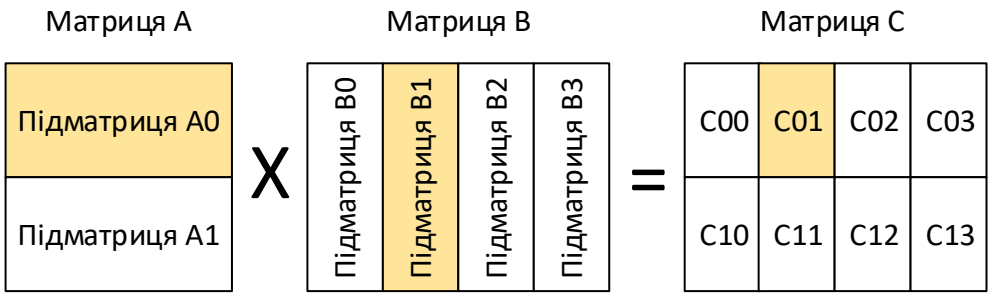

Рис. 1. Схематичне зображення поділу матриць для решітки виду 2 × 4.

Для решітки 2 × 4 матриця A ділиться на 2 смуги по рядках, а матриця B ділиться на 4 смуги по стовпцях:

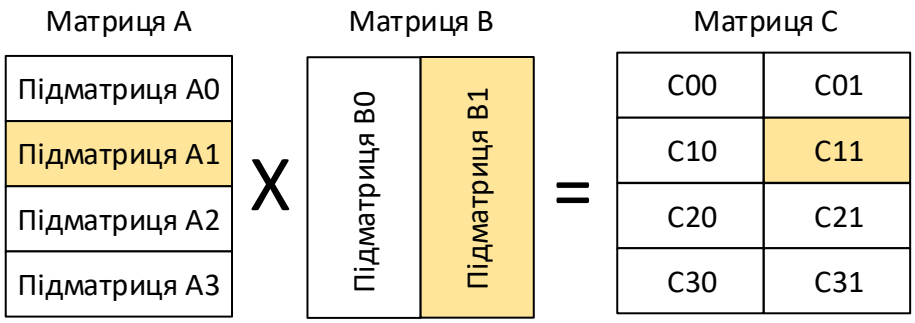

Рис. 2. Схематичне зображення поділу матриць для решітки виду 4 × 2.

Кожен процес $P(i, j)$ обчислює частину матриці C:

$$C_{ij} = A_i * B_j.$$

Для виконання множення необхідно організувати передачу смуг матриць між процесами. Найпростіший варіант – початкове розсилання смуг матриці A процесам одного рядка і смуг матриці B процесам одного стовпця. Далі кожен процес виконує обчислення своєї частини матриці C і пересилає результат процесу, який «збирає» матрицю C.

Метод множення матриць на структурі 2D решітка є одним із найбільш ефективних способів паралельного виконання цієї операції. Завдяки розподілу даних між процесами та організації обміну інформацією між рядками і стовпцями решітки можна значно скоротити час виконання обчислень.

ПОРЯДОК ВИКОНАННЯ РОБОТИ

1. Написати програму множення 2 матриць для 1-процесорної послідовної системи.
2. Написати програму множення 2 матриць на структурі 2D решітка з 8 процесорів, засобами MPI. При цьому слід дотримуватися таких вимог:
   - 2.1. Розмірності матриць та вид решітки визначаються варіантом.
   - 2.2. Результати роботи кожного процесу повинен відображатися у тестових файлах з відповідними назвами. Остаточний результат – окремо.
   - 2.3. Передбачити відображення процесу обміну даними (без відображення самих даних) на екрані.
3. Протестувати створені програми і визначити час обчислень для паралельної та послідовної системи. Збільшити розмір матриць у 2, 3, 5 і 10 раз і дослідити час обчислень в залежності від розмірності вхідних даних.
4. Зробити висновки про ефективність розпаралелення.

ЗМІСТ ЗВІТУ

1. Тема, мета, аналіз завдання.
2. Розрахунок розмірів частин матриць, що будуть опрацьовуватися окремими процесорами.
3. Результати обчислень в кожному процесорі і остаточний результат множення(скріншоти)
4. Результати дослідження ефективності паралельної програми у порівнянні з послідовною за часом виконання у табличній і графічній формі.
5. Висновки.

ВАРІАНТИ ЗАВДАНЬ

Варіант№	Вид решітки		
	N1	N2	N3
1	решітка виду 2х4		
	256	541	325
2	решітка виду 4х2		
	300	112	214
3	решітка виду 2х4		
	113	200	77
4	решітка виду 4х2		
	247	111	198
5	решітка виду 2х4		
	251	75	65
6	решітка виду 4х2		
	232	311	100
7	решітка виду 2х4		
	58	145	217
8	решітка виду 4х2		
	69	112	124
9	решітка виду 2х4		
	196	301	115
10	решітка виду 4х2		
	128	89	278
11	решітка виду 2х4		
	187	219	120
12	решітка виду 4х2		
	210	211	148
13	решітка виду 2х4		
	222	168	201
14	решітка виду 4х2		
	149	267	215
15	решітка виду 2х4		
	147	214	265
16	решітка виду 4х2		
	217	149	100
17	решітка виду 2х4		
	101	217	220
18	решітка виду 4х2		
	266	149	187
19	решітка виду 2х4		
	197	211	99

20	решітка виду 4х2		
	87	319	213
21	решітка виду 2х4		
	254	117	101
22	решітка виду 4х2		
	107	247	100
23	решітка виду 2х4		
	187	198	99
24	решітка виду 4х2		
	297	143	88
25	решітка виду 2х4		
	183	140	210
26	решітка виду 4х2		
	221	198	97
27	решітка виду 2х4		
	95	159	137
28	решітка виду 4х2		
	181	150	163
29	решітка виду 2х4		
	176	149	152
30	решітка виду 4х2		
	139	284	73

ПРИКЛАД ВИКОНАННЯ

n1 = 60, n2 = 248, n3 = 149, решітка виду 2x4

Задано дві матриці A і B. Обчислюється їх добуток C[n1][n3] = A[n1][n2] * B[n2][n3]. Матриця A попередньо розрізана на 2 смуги і матриця B попередньо розрізана на 4 смуги (обчислення будуть виконуватись у 8-ми процесах).

Отже, необхідно обчислити C[60][149] = A[60][248] * B[248][149]

Визначення розмірів підматриць (смуг):

Для матриці A поділ відбувається по рядках. Маємо: 60/2=30. Таким чином 2 підматриці A будуть мати по 30 рядків. Кількість стовпців – 248.

Для матриці B поділ відбувається по стовпцях. Маємо: 149/4=37 (залишок 1), тобто 149=3*37+1*38. Таким чином 3 підматриці B будуть мати по 37 стовпців і 1 підматриця B буде мати 38 стовпців. Кількість рядків – 248.

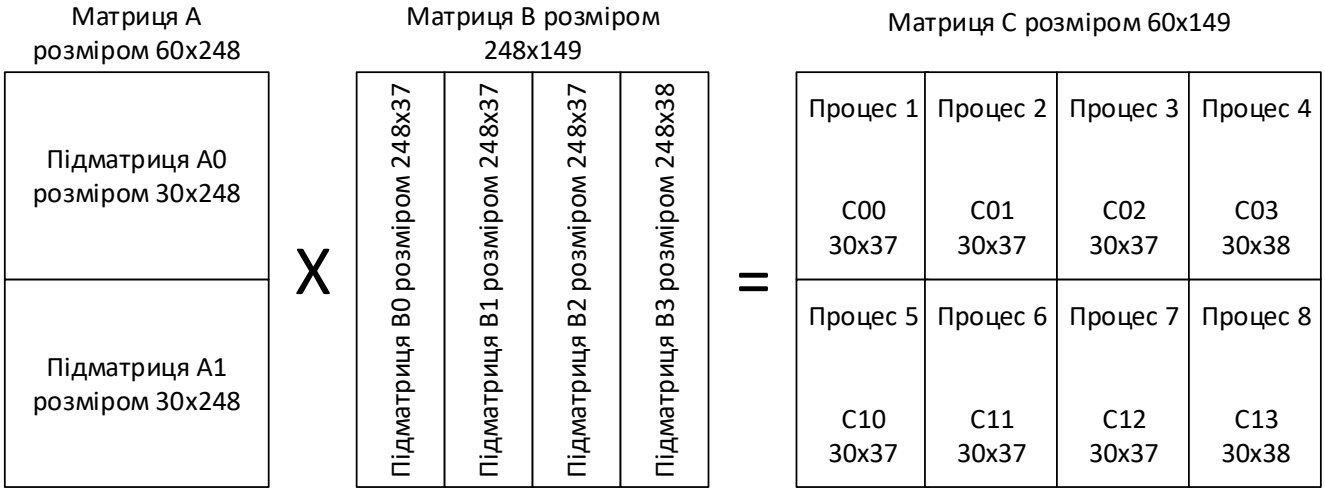

Рис. 1. Схематичне зображення поділу матриць для решітки виду 2x4

Підматриці C_{ij} для решітки виду 2x4 обчислюються таким чином:

Процес 1 обчислює $C_{00} = A_0 * B_0$
Процес 2 обчислює $C_{01} = A_0 * B_1$
Процес 3 обчислює $C_{02} = A_0 * B_2$
…
Процес 7 обчислює $C_{12} = A_1 * B_2$
Процес 8 обчислює $C_{13} = A_1 * B_3$

Обчислення будуть виконувати 8 процесів, які пов’язані між собою у структуру 2D решітка. 0-вий процес буде розсилати вхідні дані до «робочих» процесів і збирати результат в матрицю C.

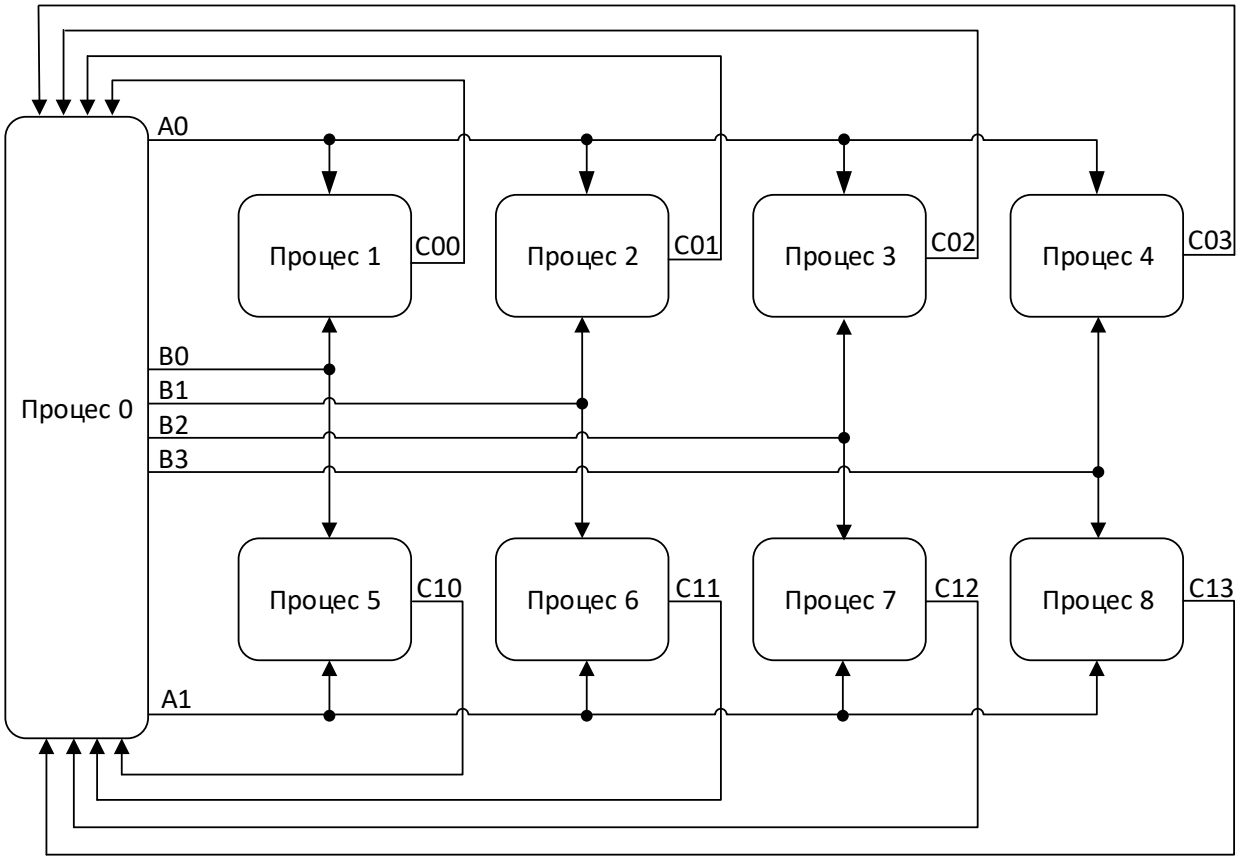

Рис. 3. Структурна схема зв’язків між процесами для решітки виду 2x4

Обчислення відбувається в такій послідовності.

1. Кожен процес отримує відповідну йому смугу матриці A. Перша смуга (підматриця A0) повинна потрапити до 1, 2, 3 і 4 процесів. Друга смуга (підматриця A1) повинна потрапити до 5, 6, 7 і 8 процесів.

2. Кожен процес отримує відповідну йому смугу матриці B. Перша смуга (підматриця B0) повинна потрапити до 1 і 5 процесів, друга (підматриця B1) повинна потрапити до 2 і 6 процесів, третя (підматриця B2) повинна потрапити до 3 і 7 процесів, четверта (підматриця B3) повинна потрапити до 4 і 8 процесів.

3. Кожен процес обчислює одну підматрицю C.

4. Матриця C збирається в нульовому процесі. Усього буде 2*4=8 частин (підматриць) C.

Визначимо час обчислень для паралельної та послідовної системи. Збільшимо розмір матриць у 2, 3, 5, 7 і 10 раз і дослідимо час обчислень в залежності від розмірності вхідних даних.

Таблиця 1.

Час виконання послідовної і MPI програми

Розмір матриць, раз	Час виконання, мс	
	Однопотокова програма	MPI програма
1	10,6879	5,8603
2	95,7837	39,0821
3	351,0731	145,8828
5	3753,1181	871,5374
7	10958,4770	2694,8283
10	35602,9303	8598,3838

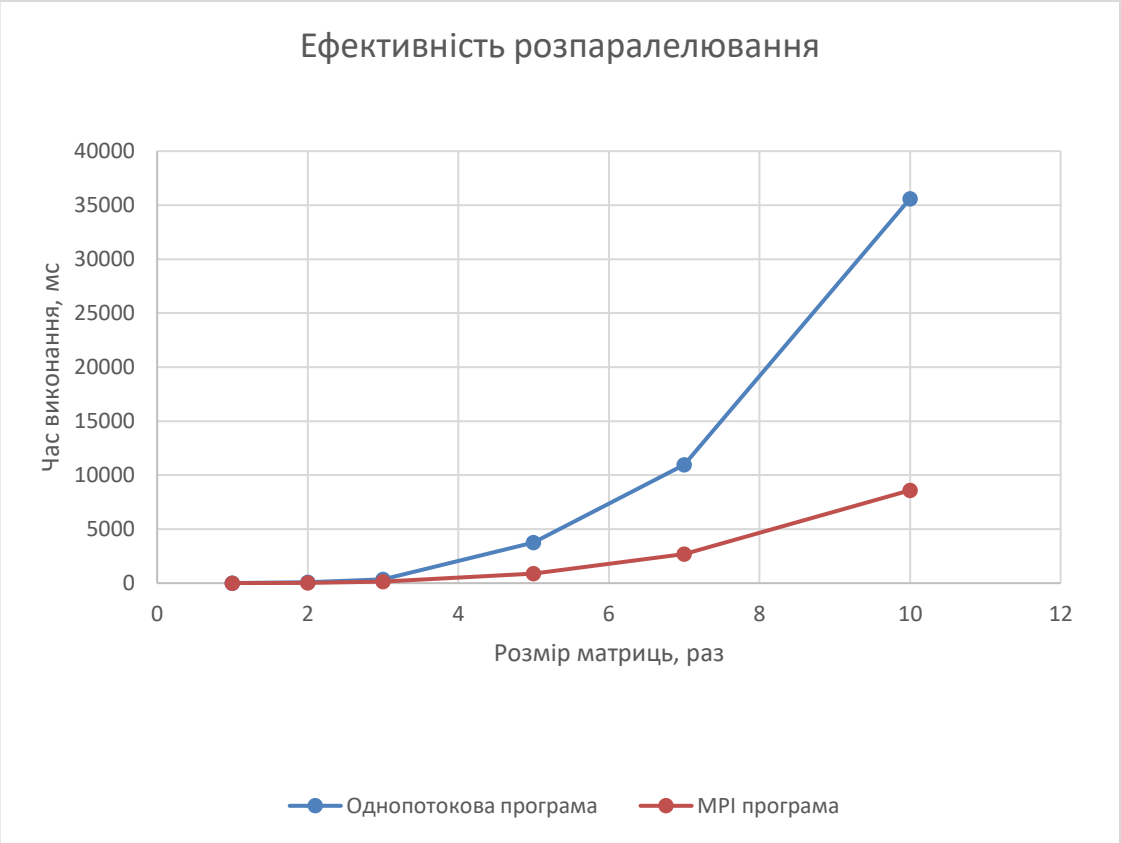

Рис. 4. Графік ефективності розпаралелювання

Висновки. Під час виконання лабораторної роботи, розроблено паралельний алгоритм перемноження матриці на матрицю на структурі 2D решітка виду 2x4 та його програмну реалізацію. Здійснено тестування програми і порівняння результатів послідовної та паралельної реалізації множення матриць показало ефективність останньої. Ефективність паралельної програми зростає з розміром матриць.

ДОДАТОК А - Результати виконання MPI-програми:

```
Output the results to the screen and to the file? 1 - yes, else - no : 1
I am 0 process. Send SubMatrix A0 to 1, 2, 3, 4 process
I am 1 process. Received SubMatrix A0 from 0 process
I am 4 process. Received SubMatrix A0 from 0 process
I am 2 process. Received SubMatrix A0 from 0 process
I am 3 process. Received SubMatrix A0 from 0 process
I am 0 process. Send SubMatrix A1 to 5, 6, 7, 8 process
I am 5 process. Received SubMatrix A1 from 0 process
I am 6 process. Received SubMatrix A1 from 0 process
I am 7 process. Received SubMatrix A1 from 0 process
I am 8 process. Received SubMatrix A1 from 0 process
I am 0 process. Send SubMatrix B0 to 1 process
I am 1 process. Received SubMatrix B0 from 0 process
I am 0 process. Send SubMatrix B1 to 2 process
I am 2 process. Received SubMatrix B1 from 0 process
I am 0 process. Send SubMatrix B2 to 3 process
I am 3 process. Received SubMatrix B2 from 0 process
I am 0 process. Send SubMatrix B3 to 4 process
I am 4 process. Received SubMatrix B3 from 0 process
I am 1 process. Multiplied SubMatrix A0 by B0 = SubMatrix C00
I am 2 process. Multiplied SubMatrix A0 by B1 = SubMatrix C01
I am 0 process. Send SubMatrix B0 to 5 process
I am 5 process. Received SubMatrix B0 from 0 process
I am 4 process. Multiplied SubMatrix A0 by B3 = SubMatrix C03
I am 0 process. Send SubMatrix B1 to 6 process
I am 6 process. Received SubMatrix B1 from 0 process
I am 3 process. Multiplied SubMatrix A0 by B2 = SubMatrix C02
I am 0 process. Send SubMatrix B2 to 7 process
I am 7 process. Received SubMatrix B2 from 0 process
I am 5 process. Multiplied SubMatrix A1 by B0 = SubMatrix C10
I am 0 process. Send SubMatrix B3 to 8 process
I am 0 process. Received SubMatrix C00 from 1 process
I am 1 process. Send SubMatrix C00 to 0 process
I am 8 process. Received SubMatrix B3 from 0 process
I am 6 process. Multiplied SubMatrix A1 by B1 = SubMatrix C11
I am 0 process. Received SubMatrix C01 from 2 process
I am 2 process. Send SubMatrix C01 to 0 process
I am 7 process. Multiplied SubMatrix A1 by B2 = SubMatrix C12
I am 8 process. Multiplied SubMatrix A1 by B3 = SubMatrix C13
I am 0 process. Received SubMatrix C02 from 3 process
I am 0 process. Received SubMatrix C03 from 4 process
I am 4 process. Send SubMatrix C03 to 0 process
I am 3 process. Send SubMatrix C02 to 0 process
I am 5 process. Send SubMatrix C10 to 0 process
I am 0 process. Received SubMatrix C10 from 5 process
I am 6 process. Send SubMatrix C11 to 0 process
I am 0 process. Received SubMatrix C11 from 6 process
I am 7 process. Send SubMatrix C12 to 0 process
I am 0 process. Received SubMatrix C12 from 7 process
I am 8 process. Send SubMatrix C13 to 0 process
I am 0 process. Received SubMatrix C13 from 8 process
```

```
// ДОДАТОК Б – Послідовна програма
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <malloc.h>
#include <time.h>
#include <windows.h>

#define K 10
#define size1 60*K
#define size2 248*K
#define size3 149*K

double** CreateMatrix(int m, int n)
{
	double** a = (double**)malloc(m * sizeof(double*));
	for (int i = 0; i < m; i++)
		a[i] = (double*)malloc(n * sizeof(double));
	return a;
}

void DeleteMatrix(double** a, int m, int n)
{
	for (int i = 0; i < m; i++)
		free(a[i]);
	free(a);
}

void FillMatrix(double** a, int m, int n)
{
	for (int i = 0; i < m; i++)
		for (int j = 0; j < n; j++)
		{
			a[i][j] = rand() % 100 + 1;
		}

}

void FillMatrixOne(double** a, int m, int n)
{
	for (int i = 0; i < m; i++)
		for (int j = 0; j < n; j++)
		{
			a[i][j] = 1;
		}

}

void PrintMatrixToFile(FILE* f, char* title, double** a, int m, int n)
{
	fprintf(f, title);
	fprintf(f, "\n");
	for (int i = 0; i < m; i++)
	{
		for (int j = 0; j < n; j++)
			fprintf(f, "%.0f\t", a[i][j]);
		fprintf(f, "\n");
	}
	fprintf(f, "\n");
}

// c = a * b, a – [m][n], b – [n][k], c – [m][k]
double** MultMatrix(double** c, double** a, double** b, int m, int n, int k)
{
	for (int i = 0; i < m; i++)
		for (int j = 0; j < k; j++)
		{
			c[i][j] = 0;
```

```
                    for (int l = 0; l < n; l++)
                          c[i][j] = c[i][j] + a[i][l] * b[l][j];
              }
      return c;
}


int main(int argc, char* argv[])
{
      srand((unsigned int)time(NULL));

      LARGE_INTEGER frequency, start, end;

      double** A = CreateMatrix(size1, size2);
      double** B = CreateMatrix(size2, size3);
      double** C = CreateMatrix(size1, size3);

      FILE* file = fopen("Results.txt", "w");
      if (file == NULL)
      {
            printf("\nError opening file. Check the path and permissions !\n");
            return 1;
      }

      // random input matrix A
      FillMatrix(A, size1, size2);
      PrintMatrixToFile(file, (char*)"Matrix A = ", A, size1, size2);

      // random input matrix B
      FillMatrix(B, size2, size3);
      PrintMatrixToFile(file, (char*)"Matrix B = ", B, size2, size3);

      QueryPerformanceFrequency(&frequency);
      QueryPerformanceCounter(&start);

      MultMatrix(C, A, B, size1, size2, size3);

      QueryPerformanceCounter(&end);
      double elapsedTime = static_cast<double>(end.QuadPart - start.QuadPart) /
frequency.QuadPart * 1000;
      printf("\nElapsed Time: %f milliseconds\n\n", elapsedTime);

      PrintMatrixToFile(file, (char*)"Matrix C = ", C, size1, size3);

      DeleteMatrix(A, size1, size2);
      DeleteMatrix(B, size2, size3);
      DeleteMatrix(C, size1, size3);

      fclose(file);

      return 0;
}
```

```
//ДОДАТОК В – MPI програма
#define _CRT_SECURE_NO_WARNINGS
#include "mpi.h"
#include <stdio.h>
#include <stdlib.h>
#include <malloc.h>
#include <time.h>
#include <windows.h>

#define K 1
#define size1 60*K
#define size2 248*K
#define size3 149*K


double** CreateMatrix(int m, int n)
{
	double** a = (double**)malloc(m * sizeof(double*));
	for (int i = 0; i < m; i++)
		a[i] = (double*)malloc(n * sizeof(double));
	return a;
}

void DeleteMatrix(double** a, int m, int n)
{
	for (int i = 0; i < m; i++)
		free(a[i]);
	free(a);
}

void FillMatrix(double** a, int m, int n)
{
	for (int i = 0; i < m; i++)
		for (int j = 0; j < n; j++)
		{
			a[i][j] = rand() % 100 + 1;
		}

}

void FillMatrixOne(double** a, int m, int n)
{
	for (int i = 0; i < m; i++)
		for (int j = 0; j < n; j++)
		{
			a[i][j] = 1;
		}

}

void PrintMatrixToFile(FILE* f, char* title, double** a, int m, int n)
{
	fprintf(f, title);
	fprintf(f, "\n");
	for (int i = 0; i < m; i++)
	{
		for (int j = 0; j < n; j++)
			fprintf(f, "%.0f\t", a[i][j]);
		fprintf(f, "\n");
	}
	fprintf(f, "\n");
}

// c = a * b, a – [m][n], b – [n][k], c – [m][k]
double** MultMatrix(double** c, double** a, double** b, int m, int n, int k)
{
	for (int i = 0; i < m; i++)
		for (int j = 0; j < k; j++)
```

```
		{
			c[i][j] = 0;
			for (int l = 0; l < n; l++)
				c[i][j] = c[i][j] + a[i][l] * b[l][j];
		}
	return c;
}

int main(int argc, char* argv[])
{
	srand((unsigned int)time(NULL));

	MPI_Init(NULL, NULL);

	LARGE_INTEGER frequency, start, end;

	int rank, size;

	MPI_Comm_rank(MPI_COMM_WORLD, &rank);
	MPI_Comm_size(MPI_COMM_WORLD, &size);

	if (size != 9)
	{
		printf("This program requires 9 processes !\n");
		fflush(stdout);
		MPI_Abort(MPI_COMM_WORLD, 1);
	}

	int i, j, k;
	int out = 0;

	char SubMatrix[32];
	char fileName[32];

	MPI_Comm commA0, commA1;

	MPI_Comm_split(MPI_COMM_WORLD, (rank <= 4) ? 1 : MPI_UNDEFINED, rank, &commA0);
	MPI_Comm_split(MPI_COMM_WORLD, (rank == 0 || rank >= 5) ? 1 : MPI_UNDEFINED, rank,
&commA1);


	if (rank == 0)
	{
		double** A = CreateMatrix(size1, size2);
		double** B = CreateMatrix(size2, size3);
		double** C = CreateMatrix(size1, size3);

		sprintf(fileName, "Results_0.txt");
		FILE* file = fopen(fileName, "w");
		if (file == NULL)
		{
			printf("\nError opening file. Check the path and permissions !\n");
			return 1;
		}

		printf("Output the results to the screen and to the file? 1 - yes, else - no
: ");
		fflush(stdout);
		scanf("%d", &out);


		MPI_Bcast(&out, 1, MPI_INT, 0, MPI_COMM_WORLD);

		// random input matrix A
		FillMatrix(A, size1, size2);
		PrintMatrixToFile(file, (char*)"Matrix A = ", A, size1, size2);

		// random input matrix B
```

```
		FillMatrix(B, size2, size3);
		PrintMatrixToFile(file, (char*)"Matrix B = ", B, size2, size3);

		QueryPerformanceFrequency(&frequency);
		QueryPerformanceCounter(&start);

		// розсилання підматриці A0
		for (j = 0; j < 30 * K; j++)
			MPI_Bcast(A[j], size2, MPI_DOUBLE, 0, commA0);

		if (out == 1)
		{
			printf("I am 0 process. Send SubMatrix A0 to 1, 2, 3, 4 process\n");
			fflush(stdout);
		}

		// розсилання підматриці A1
		for (j = 0; j < 30 * K; j++)
			MPI_Bcast(A[j + 30 * K], size2, MPI_DOUBLE, 0, commA1);

		if (out == 1)
		{
			printf("I am 0 process. Send SubMatrix A1 to 5, 6, 7, 8 process\n");
			fflush(stdout);
		}

		// розсилання підматриць B
		int B_columns;
		int shift;

		for (i = 1; i <= 8; i++)
		{

			if (i == 4 || i == 8)
				B_columns = 38 * K;
			else
				B_columns = 37 * K;

			MPI_Send(&B_columns, 1, MPI_INT, i, 0, MPI_COMM_WORLD);

			if (i == 5 || i == 1)
				shift = 0;

			for (j = 0; j < size2; j++)
				MPI_Ssend((B[j] + shift), B_columns, MPI_DOUBLE, i, 0,
MPI_COMM_WORLD);

			shift += B_columns;

			if (out == 1)
			{
				printf("I am 0 process. Send SubMatrix B%d to %d process\n", (i
- 1) % 4, i);
				fflush(stdout);
			}

		}

		// отримання частин матриці C
		for (int i = 1; i <= 8; i++)
		{
			MPI_Recv(&B_columns, 1, MPI_INT, i, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);

			if (i <= 4)
			{
				j = 0;
				shift = (i - 1) * 37;
```

```
                }
                else
                {
                        j = 30 * K;
                        shift = (i - 5) * 37;
                }

                for (k = 0; k < 30 * K; k++, j++)
                        MPI_Recv((C[j] + shift), B_columns, MPI_DOUBLE, i, 0,
MPI_COMM_WORLD, MPI_STATUS_IGNORE);

                if (out == 1)
                {
                        printf("I am 0 process. Received SubMatrix C%d%d from %d
process\n", (i - 1) / 4, (i - 1) % 4, i);
                        fflush(stdout);
                }
        }

        QueryPerformanceCounter(&end);
        double elapsedTime = static_cast<double>(end.QuadPart - start.QuadPart) /
frequency.QuadPart * 1000;
        printf("\nElapsed Time: %f milliseconds\n\n", elapsedTime);

        PrintMatrixToFile(file, (char*)"Matrix C = ", C, size1, size3);

        DeleteMatrix(A, size1, size2);
        DeleteMatrix(B, size2, size3);
        DeleteMatrix(C, size1, size3);

        fclose(file);
    }
    else
    {

        int I = (rank - 1) / 4;
        int J = (rank - 1) % 4;

        sprintf(fileName, "Results_%d.txt", rank);

        FILE* file = fopen(fileName, "w");
        if (file == NULL)
        {
                printf("Error opening file. Check the path and permissions !\n");
                return 1;
        }

        MPI_Bcast(&out, 1, MPI_INT, 0, MPI_COMM_WORLD);

        // отримання поточним процесом смуги матриці A
        double** a = CreateMatrix(30 * K, size2);

        if (rank <= 4)
                for (j = 0; j < 30 * K; j++)
                        MPI_Bcast(a[j], size2, MPI_DOUBLE, 0, commA0);
        else
                for (j = 0; j < 30 * K; j++)
                        MPI_Bcast(a[j], size2, MPI_DOUBLE, 0, commA1);

        if (out == 1)
        {
                printf("I am %d process. Received SubMatrix A%d from 0 process\n",
rank, I);
                fflush(stdout);

                sprintf(SubMatrix, "SubMatrix A%d =  ", I);
                PrintMatrixToFile(file, SubMatrix, a, 30 * K, size2);
        }
```

```
            // отримання поточним процесом смуги матриці B
            double** b = CreateMatrix(size2, 38 * K);

            int B_columns;
            MPI_Recv(&B_columns, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
            for (i = 0; i < size2; i++)
                  MPI_Recv(b[i], B_columns, MPI_DOUBLE, 0, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);

            if (out == 1)
            {
                  printf("I am %d process. Received SubMatrix B%d from 0 process\n",
rank, J);
                  fflush(stdout);

                  sprintf(SubMatrix, "SubMatrix B%d =  ", J);
                  PrintMatrixToFile(file, SubMatrix, b, size2, B_columns);
            }

            double** c = CreateMatrix(30 * K, 38 * K);

            // множення смуги матриці A на смугу матриці B
            MultMatrix(c, a, b, 30 * K, size2, B_columns);

            if (out == 1)
            {
                  printf("I am %d process. Multiplied SubMatrix A%d by B%d = SubMatrix
C%d%d \n", rank, I, J, I, J);
                  fflush(stdout);
            }

            // надсилання частини матриці C 0-му процесу
            MPI_Send(&B_columns, 1, MPI_INT, 0, 0, MPI_COMM_WORLD);
            for (i = 0; i < 30 * K; i++)
                  MPI_Ssend(c[i], B_columns, MPI_DOUBLE, 0, 0, MPI_COMM_WORLD);

            if (out == 1)
            {
                  printf("I am %d process. Send SubMatrix C%d%d to 0 process\n", rank,
I, J);
                  fflush(stdout);

                  sprintf(SubMatrix, "SubMatrix C%d%d = A%d x B%d ", I, J, I, J);
                  PrintMatrixToFile(file, SubMatrix, c, 30 * K, B_columns);
            }

            DeleteMatrix(a, 30 * K, size2);
            DeleteMatrix(b, size2, 38 * K);
            DeleteMatrix(c, 30 * K, 38 * K);

            fclose(file);
      }

      MPI_Finalize();
      return 0;
}
```